

Checkpoint 1

Background, Experimental Design, Forward Plan, and Anticipated Technical Risks

Project: Rank-Adaptive and Sparsity-Aware Dynamic Quadtree Matrix Multiplication over Containerized Microservices

Course Option: CSC 4770 Option 1 — Graduate / 5770 Requirements

Checkpoint Focus: Background, experimental design, implementation path, and anticipated areas of contention.

1. Checkpoint Purpose

The first checkpoint is intended to establish that the project has a technically defensible direction before full implementation begins.

At this stage, the project should be able to answer four questions:

1. **What problem is being studied?**
2. **Why is the problem technically meaningful?**
3. **How will the proposed system be evaluated?**
4. **What risks or implementation issues may affect the final result?**

The project is not being framed as an attempt to invent a universally faster matrix multiplication algorithm. Instead, it is an experimental systems study of how numerical structure interacts with communication overhead in a containerized distributed environment.

2. Project Background

Matrix multiplication is a foundational operation in scientific computing, machine learning, numerical simulation, graphics, and high-performance computing.

For two square matrices

$$A, B \in \mathbb{R}^{N \times N},$$

the product

$$C = AB$$

is defined elementwise by

$$C_{ij} = \sum_{k=1}^N A_{ik} B_{kj}.$$

The conventional dense algorithm requires cubic-order work:

$$O(N^3).$$

The computation is naturally decomposable because output regions can be expressed as sums of independent block products.

If the matrices are divided into $q \times q$ blocks,

$$C_{ij} = \sum_{k=1}^q A_{ik} B_{kj}.$$

Each partial block multiplication

$$T_{ikj} = A_{ik} B_{kj}$$

can therefore be treated as an independent task until the aggregation stage.

This structure makes matrix multiplication suitable for parallel and distributed experimentation.

3. Why a Containerized Microservice Implementation Is Interesting

Traditional HPC matrix multiplication often assumes tightly coupled processes, optimized numerical libraries, and high-bandwidth interconnects.

This project intentionally studies a different environment:

- Docker containers,
- isolated service roles,
- TCP sockets,
- application-layer framing,
- software bridge networking,
- explicit serialization,
- independently scheduled worker processes.

That environment introduces costs that are less visible in a shared-memory program.

Examples include:

- network transmission,
- socket framing,
- serialization,
- task scheduling,
- message latency,
- container resource contention,
- result aggregation.

As a result, total execution time cannot be modeled as computation alone.

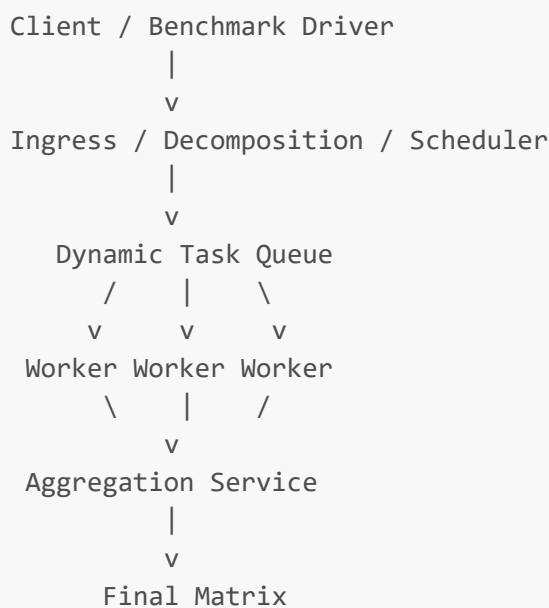
A practical decomposition is

$$T_{\text{total}} = T_{\text{ingress}} + T_{\text{queue}} + T_{\text{comm}} + T_{\text{worker}} + T_{\text{aggregation}}.$$

The project will instrument these components independently.

4. Baseline Distributed Design

The required system is organized into three primary microservice roles.



The Ingress service is responsible for:

- validating matrix input,
- padding dimensions when necessary,
- decomposing matrices into blocks,
- generating block-multiplication tasks,
- scheduling tasks across workers.

Workers independently execute block multiplications.

The Aggregation service combines partial results using

$$C_{ij} \leftarrow C_{ij} + T_{ikj}.$$

The final product is validated against a trusted serial result.

5. Quadtree-Compatible Decomposition

A binary quadtree requires repeated halving until the configured leaf block size is reached.

To ensure this is always possible, the padded dimension is defined by

$$d = \max \left(0, \left\lceil \log_2 \left(\frac{N}{b_{\text{leaf}}} \right) \right\rceil \right),$$

followed by

$$N' = b_{\text{leaf}} 2^d.$$

This guarantees the recursion

$$N' \rightarrow \frac{N'}{2} \rightarrow \frac{N'}{4} \rightarrow \dots \rightarrow b_{\text{leaf}}.$$

After the distributed multiplication is complete, the padded result is sliced back to the original dimension.

6. Communication as a Primary Experimental Variable

Let the leaf block dimension be b , and define

$$q = \frac{N'}{b}.$$

There are

$$q^3$$

leaf block-multiplication tasks.

For the dense baseline, each task requires two $b \times b$ float64 inputs.

The input payload per task is

$$16b^2$$

bytes.

Therefore,

$$D_{\text{dispatch}} = 16q^3b^2 = 16\frac{N'^3}{b}.$$

If each worker returns one dense $b \times b$ partial product,

$$D_{\text{return}} = 8\frac{N'^3}{b}.$$

Ignoring framing bytes,

$$D_{\text{dense,total}} = 24\frac{N'^3}{b}.$$

This relationship is important because it establishes an immediate systems tradeoff.

Smaller blocks increase available task parallelism, but they also increase:

- task count,
- message count,
- scheduler overhead,
- total data movement.

One major experimental question is therefore:

Which block sizes produce useful parallelism without creating excessive communication overhead?

7. Adaptive Extension 1: Approximate Sparsity Pruning

The first proposed optimization attempts to avoid transmitting block products that are numerically negligible.

For a candidate task

$$T_{ikj} = A_{ik}B_{kj},$$

the Ingress service computes

$$\|A_{ik}\|_F \quad \text{and} \quad \|B_{kj}\|_F.$$

Because

$$\|A_{ik}B_{kj}\|_F \leq \|A_{ik}\|_F \|B_{kj}\|_F,$$

a task may be pruned when

$$\|A_{ik}\|_F \|B_{kj}\|_F \leq \epsilon_{\text{zero}}.$$

The system substitutes

$$\hat{T}_{ikj} = 0.$$

This must be treated as an approximation when

$$\epsilon_{\text{zero}} > 0.$$

The local error is bounded by

$$\|T_{ikj} - \hat{T}_{ikj}\|_F \leq \epsilon_{\text{zero}}.$$

The final system error will be measured using

$$\text{Relative Error} = \frac{\|C - \hat{C}\|_F}{\|C\|_F}.$$

This gives the project a direct performance-versus-accuracy measurement.

8. Adaptive Extension 2: Low-Rank Factored Transmission

Some matrix blocks may contain strong numerical redundancy even when they are not sparse.

A block can be approximated using truncated factorization:

$$A \approx U\Sigma V^T.$$

Let

$$X = U\Sigma, \quad Y = V.$$

Then

$$A \approx XY^T.$$

For a $b \times b$ block of numerical rank r , the dense representation requires

$$8b^2$$

bytes, while the two factor matrices require approximately

$$16br$$

bytes.

The payload ratio is therefore

$$\frac{D_{\text{factor}}}{D_{\text{dense}}} = \frac{2r}{b}.$$

When

$$r \ll b,$$

factored transmission may substantially reduce network volume.

The worker can also exploit the factorization algebraically.

If

$$A \approx X_A Y_A^T,$$

then

$$AB \approx X_A (Y_A^T B),$$

which can reduce arithmetic cost from approximately

$$O(b^3)$$

to

$$O(b^2 r_A).$$

If both operands are low-rank,

$$A \approx X_A Y_A^T$$

and

$$B \approx X_B Y_B^T,$$

then

$$AB \approx X_A (Y_A^T X_B) Y_B^T.$$

This creates the possibility of reducing both communication and worker compute cost.

9. Important Low-Rank Risk

Low-rank detection itself is not free.

A major concern is that

$$T_{\text{factorization}} > T_{\text{communication saved}}.$$

If the Ingress service performs expensive matrix factorization on every block, preprocessing may become the dominant bottleneck.

The project will therefore treat rank-adaptive transmission as a later optimization layer.

Planned controls include:

- truncated or randomized factorization,
- minimum block size for rank analysis,
- maximum accepted rank r_{max} ,
- relative reconstruction tolerance ϵ_{rank} ,
- predicted byte-savings threshold,
- separate measurement of factorization time.

A negative result is still meaningful if the experiment identifies matrix classes for which low-rank analysis does not amortize.

10. Experimental Systems

The experiment will compare at least three implementations.

10.1 Serial Baseline

Single-process dense multiplication.

Purpose:

- correctness reference,
- baseline runtime,
- source for speedup calculations.

10.2 Static Docker Baseline

Distributed block multiplication using:

- Docker workers,
- TCP,
- dense block transmission,
- no sparsity pruning,
- no low-rank compression.

Purpose:

- isolate containerization and parallelism costs.

10.3 Adaptive Docker System

The same distributed architecture plus:

- dynamic work scheduling,
- approximate sparsity pruning,
- optional rank-adaptive transmission.

Purpose:

- isolate the contribution of the proposed adaptive mechanisms.

This three-way comparison prevents the project from incorrectly attributing all speedup to the adaptive algorithm.

11. Matrix Families

Using only random dense matrices would not adequately test the proposed optimizations.

The benchmark set will contain several matrix families.

Dense Random

Purpose:

- control,
- approximately full-rank,
- non-sparse,
- expected worst case for adaptive preprocessing.

Block Sparse

Purpose:

- test pruning behavior,
- measure skipped tasks and network savings.

Low-Rank

Constructed in reproducible form, for example

$$A = XY^T,$$

where

$$r \ll N.$$

Purpose:

- test rank detection,
- measure factor-transmission savings.

Mixed Structured

Contains:

- dense blocks,
- sparse blocks,
- low-rank blocks.

Purpose:

- exercise dynamic scheduling and heterogeneous task costs.

12. Independent Variables

The planned experiments will vary some or all of the following:

Variable	Candidate Values
Matrix size N	100, 500, 1000, 2000, 5000, up to 10000
Workers P	1, 2, 4, 8
Leaf block size b_{leaf}	128, 256, 512, configurable
Matrix family	dense, sparse, low-rank, mixed
ϵ_{zero}	disabled plus selected tolerances
ϵ_{rank}	selected tolerance sweep
r_{max}	configurable

The full Cartesian product will not necessarily be run. The benchmark matrix will be reduced if necessary to keep the experiment tractable.

13. Metrics

The system will collect four categories of metrics.

Performance

$$T_P$$

$$S_P = \frac{T_1}{T_P}$$

$$E_P = \frac{S_P}{P}$$

plus:

- total wall-clock time,
- preprocessing time,
- queue wait time,
- communication time,
- worker compute time,
- aggregation time.

Communication

- total bytes sent,
- total bytes returned,
- number of messages,
- tasks created,
- tasks dispatched,
- tasks pruned,
- dense tasks,
- factored tasks,
- compression ratio.

Accuracy

$$\frac{\|C - \hat{C}\|_F}{\|C\|_F}$$

with the serial implementation used as the numerical reference.

Scheduling

- worker utilization,

- worker idle time,
- queue depth,
- per-task execution time,
- task retries.

14. Dynamic Scheduling Strategy

A static assignment such as

```
Worker 1 -> Tasks 1-100  
Worker 2 -> Tasks 101-200  
...
```

may perform poorly because adaptive tasks can have very different execution costs.

The preferred design is a central task queue:

$$Q = \{T_1, T_2, \dots, T_n\}.$$

Workers request another task whenever they become idle.

This pull-based strategy should:

- reduce worker idle time,
- improve load balance,
- handle pruned and factored tasks naturally,
- prevent one worker from becoming a persistent bottleneck.

15. TCP and Serialization Strategy

The system will use persistent TCP connections where practical.

A fresh TCP connection should not be created for every matrix block.

Each message will use application-layer binary framing because TCP does not preserve message boundaries.

The receiver will:

1. read the fixed header,
2. validate the message,
3. read exactly the declared payload length,
4. decode the payload according to its task type.

Dense numerical payloads will use contiguous binary `float64` data rather than JSON or XML.

This design reduces text parsing and unnecessary serialization overhead.

16. Experimental Controls

To make results reproducible:

- matrix generators will use fixed seeds,
- hardware and software versions will be recorded,
- Docker images will be built before timed trials,
- containers will be started before timed trials,
- persistent worker connections will be established before measurements,
- warm-up trials will precede timed trials,
- experiments will be repeated,
- results will report central tendency and variation,
- correctness will be checked after every run.

The experiment will distinguish computation from infrastructure startup time.

17. Forward Implementation Plan

Phase 1 — Serial Reference

Build a trusted matrix multiplication and validation path.

Phase 2 — Static Docker/TCP Prototype

Implement:

- Ingress,
- worker,
- Aggregator,
- framed TCP protocol,
- dense block multiplication.

Phase 3 — Multi-Worker Scheduler

Add:

- configurable worker count,
- persistent connections,
- dynamic pull-based work queue,
- task timing.

Phase 4 — Quadtree-Compatible Padding and Decomposition

Support arbitrary matrix sizes through

$$N' = b_{\text{leaf}} 2^d.$$

Phase 5 — Sparsity Pruning

Add:

- Frobenius bound,
- ϵ_{zero} ,
- pruning metrics,
- error metrics.

Phase 6 — Benchmark Instrumentation

Collect:

- timings,
- bytes,
- messages,
- tasks,
- speedup,
- efficiency,
- error.

At this point the project already has a complete graduate-level experimental baseline.

Phase 7 — Low-Rank Compression

Add:

- truncated/randomized factorization,
- dense/factored payload types,
- low-rank worker kernels.

Phase 8 — Experimental Campaign

Run controlled benchmark families and produce paper-ready results.

18. Expected Hiccups and Areas of Contention

18.1 Ingress Bottleneck

The Ingress service may become overloaded because it performs:

- decomposition,
- norm calculations,
- rank analysis,
- task scheduling.

This will be measured explicitly.

18.2 Rank Analysis May Not Amortize

Low-rank factorization may cost more than the network traffic it saves.

This is expected to depend strongly on:

- block size,
- numerical rank,
- network performance,
- worker count.

18.3 Block Size Tradeoff

Small blocks increase available concurrency but also increase

$$q^3$$

task count and communication overhead.

Large blocks reduce message count but may limit parallelism.

This tradeoff will be an experimental focus.

18.4 Approximation Error

Pruning and low-rank compression may produce

$$\widehat{C} \neq C.$$

The project will not describe approximate results as exact.

All performance improvements will be reported together with numerical error.

18.5 Memory Pressure

A $10,000 \times 10,000$ float64 matrix contains 100,000,000 elements and requires approximately 800 MB of raw storage.

Multiple matrices, temporary buffers, block copies, factor matrices, and Docker processes can therefore create substantial memory pressure.

The largest matrix sizes will be treated as stress benchmarks rather than routine development inputs.

18.6 TCP Stream Handling

TCP does not guarantee that one `send()` corresponds to one `recv()`.

Incorrect framing could cause:

- partial reads,
- concatenated messages,
- corrupted task boundaries.

The protocol therefore requires explicit lengths and bounded receive loops.

18.7 Worker Failure

A worker disconnect can leave a block accumulation incomplete.

The scheduler will track task states and may retry failed tasks on another worker.

18.8 Scope Expansion

The project already exceeds the minimum course implementation.

To protect deliverability, the required completion order is:

Correctness → Distributed Baseline → Instrumentation → Sparsity → Low Rank.

Low-rank compression will not be allowed to block completion of the core system.

19. What We Expect to Learn

The project is designed to identify **regimes of usefulness**, not to assume that the adaptive system always wins.

Possible findings include:

Dense Random Matrices

$$T_{\text{adaptive}} > T_{\text{static}}$$

because structure detection adds overhead without yielding useful compression.

Sparse Matrices

$$T_{\text{adaptive}} < T_{\text{static}}$$

because tasks are removed before serialization and worker execution.

Low-Rank Matrices

The adaptive system may reduce network volume if

$$r \ll b$$

and factorization overhead is sufficiently small.

Excessive Worker Counts

Parallel efficiency may fall as

$$P$$

increases because communication, scheduling, and aggregation costs begin to dominate.

These are all valid and useful outcomes.

20. Checkpoint 1 Talking Points

For the checkpoint discussion, the team should be prepared to explain the following without relying on the paper.

Background

- why matrix multiplication decomposes into independent block products,
- why Docker/TCP changes the performance model,
- why communication volume matters,
- why numerical sparsity and low rank are possible optimization targets.

Experimental Design

- the three implementations being compared,
- the matrix families,
- the independent variables,
- the metrics,
- the correctness and error methodology.

Forward Plan

- serial baseline first,
- static distributed implementation second,
- instrumentation before advanced optimization,
- sparsity pruning before low-rank compression.

Known Risks

- SVD/factorization overhead,
- Ingress bottleneck,
- block-size tradeoff,
- TCP message handling,
- approximation error,
- memory usage at large N ,
- worker load imbalance,
- project scope.

21. Current Project Position

The project currently has:

- a defined distributed architecture,
- a formal matrix decomposition model,
- a corrected quadtree padding rule,
- a corrected communication-volume model,

- a bounded approximation rule for pruning,
- a proposed low-rank transmission model,
- a dynamic scheduling strategy,
- an experimental baseline structure,
- a staged implementation plan.

The next priority is implementation of the **correct static distributed baseline**.

The advanced adaptive mechanisms should be added only after the baseline system is reproducibly correct and measurable.

22. Preliminary Conclusion

The central thesis of the project is:

In a containerized distributed matrix multiplication system, computation is only one component of runtime. Communication, serialization, task scheduling, and numerical structure can significantly alter scaling behavior.

The proposed adaptive system will test whether sparse and low-rank matrix blocks can reduce those distributed-system costs sufficiently to justify their preprocessing overhead.

The project will be considered successful even if the adaptive approach helps only specific matrix classes, because identifying those conditions is itself an experimentally meaningful result.

23. References To Be Added During Background Research

The final graduate paper requires a formal reference set. The Checkpoint 1 draft should next be expanded with peer-reviewed or authoritative sources covering:

1. blocked / tiled matrix multiplication,
2. divide-and-conquer matrix algorithms,
3. parallel matrix multiplication,
4. Cannon's algorithm or comparable distributed baselines,
5. containerized HPC performance,
6. TCP/socket communication overhead,
7. sparse matrix computation,
8. low-rank matrix approximation,
9. randomized SVD / randomized numerical linear algebra,
10. performance modeling of distributed numerical systems.

No references are inserted here yet because they should be researched and verified rather than invented.